Predictive Collision Severity System: Using Machine Learning to Forecast Time-to-Collision (TTC) and Impact Force from the FIUS Data

1st Priyanshu Bandyopadhyay
Master of Engineering - Information Technology
Frankfurt University of Applied Sciences
1565216 - priyanshu.bandyopadhyay@stud.fra-uas.de

2nd Aditya Praveen Shidhaye
Master of Engineering - Information Technology
Frankfurt University of Applied Sciences
1566213 - aditya.shidhaye@stud.fra-uas.de

3nd Pradeep Buddhiram Patwa
Master of Engineering - Information Technology
Frankfurt University of Applied Sciences
1564727 - pradeep.patwa@stud.fra-uas.de

4nd Akshay Ganesh Gudekar
Master of Engineering - Information Technology
Frankfurt University of Applied Sciences
1564842 - akshay.gudekar@stud.fra-uas.de

***Abstract*—Most ultrasonic sensors only measure distance and trigger an alarm. This project is present to addresses the problem of delayed alerts which only trigger when an object is dangerously close, often too late to react. The raw accoustic waveform is analysed instead of sensor's processed distance data. This allows it to determine how fast an object approaches, what material it is made of, and precisely when and how forcefully it will impact, all before contact occurs. The system is utilising a Kalman filter for smooth tracking and a machine learning pipeline to classify direction of movement and estimate collision timing. It can also identify the approaching object's material to assess impact force. Tested with real hand-motion recordings across three material types, it detects approaching objects with an F1-score over 0.92, predicts impact within 0.15 seconds on average, and processes each frame in under 1 millisecond, over 150 times faster than the sensor's update rate- making it well-suited for use on affordable embedded hardware.**

***Index Terms*—ultrasonic sensing, time-of-flight, Kalman filter, one-class classification, support vector regression, time-to-collision, anomaly detection, edge AI, collision prediction, signal processing**

I. Introduction

Despite their widespread use in robotics and industrial safety, standard ultrasonic sensors are surprisingly limited. Most off-the-shelf units simply report a distance value and trigger an alarm when a threshold is hit. This is a reactive approach that often fails when objects move quickly. In common deployments, sensor output is often reduced to a one dimensional scalar, i.e a single distance reading, discarding everything else the waveform contains. While this allows a microcontroller to activate actuators based on predefined thresholds, the resulting system remains blind to the object's velocity and the severity of an impending impact.

This reactive approach has two main flaws. First, it offers no early warning because the alarm only triggers when an object hits the threshold, not before. For a person moving a hand at 0.5–1.0 m/s toward a machine, a 5 cm safety zone provides less than 100 ms to react too fast for any mechanical or human response. Second, these systems treat all objects the same. A 70 kg person and a 0.5 kg piece of cardboard trigger the exact same alarm, even though the actual impact force differs by a factor of 100.

The raw ADC data produced by an ultrasonic front end at its full sampling rate contains considerably more information than the processed distance figure that firmware exposes. Each frame records the complete time domain waveform of the transmitted pulse, its propagation through the medium, and the reflected echo's shape, amplitude, and timing. From this waveform one can recover not just distance, but velocity through finite differencing or Kalman smoothing, object surface properties through echo shape analysis, and spectral signatures through Fourier transformation. The central hypothesis of this work is that fusing these engineered features enables the training of lightweight machine learning models that can accurately forecast the moment and force of an imminent collision.

To build a truly proactive system, our implementation focuses on four key stages of the signal pipeline. First, we move past basic firmware distance readings to detect the 'first echo' directly from raw ADC frames. From there, we classify motion direction, predict the Time-to-Collision (TTC) using physics-informed models, and finally estimate impact force based on the material's acoustic signature.

1) **First-echo detection:** The main goal is to identify the sample index of the earliest or first reflection within each ADC frame. Reliability is essential even for the targets in sensor's near-field blind spot because this data point serves as the basis for all following feature derivation.
2) **Motion direction classification.** The processing pipeline must decide frame-by-frame, whenever an object is approaching the sensor. To handle the messy nature of hand-held data, the system is designed to filter out measurement any jitter, noise and sudden spikes in velocity

of the object to have robust classification.

3) **Time-to-Collision prediction.** Here The main objective is to calculate the remaining time before an object reaches the sensor. Instead of a linear estimate, a physics-informed regression model factors is used in both velocity and acceleration to improve accuracy in dynamic environments.
4) **Impact force estimation.** The system identifies surface material of an approaching object from it's generated acoustic echo signature or waves. By combining this classification with the predicted kinematic state, the algorithm estimates expected impact force in Newtons. This calculation uses Newton's second law ($F = ma$), with mass being inferred directly from the identified material class.

None of the individual components here are new — Kalman filters, OCSVM, and SVR are all textbook material. What took us time to work out was how to wire them together without consistency errors between training and inference, and how to generate physically meaningful ground truth from a self-recorded dataset that has no external reference labels. The specific choices that are ours:

Dynamic Blanking Zone: Rather than applying a fixed post transmit free window, given design uses Kalman estimated distance to dynamically reduce window size as an object approaches. Which prevents a common failure in fixed blanking systems where track is lost in final critical centimetres or distance. Just by shrinking the window, the system is able to maintain visibility of the target throughout the entire process.

Quad Scale Temporal Feature Engineering: Motion is captured across four simultaneous time horizons (5, 10, 25, and 50 frames) only by computing rolling echo index trends, means, and spectral centroid trends at each scale. This approach enables the classifier to respond to both instantaneous velocity spikes and sustained slowcreep approach profiles within a single, unified feature vector.

Physics Based Ground Truth Generation: Training labels are not derived from simple velocity sign comparisons. But rather, a full quadratic kinematic solver ($\frac{1}{2}at^2 + vt + d = 0$) is applied to each approaching frame to compute a physically precise TTC ground truth. The Boundary erosion further cleans the labels by discarding ambiguous transition frames at the start and end of each movement burst.

Three Stage Cascaded Inference Engine: The final runtime system operates as a sequential pipeline: a One-Class SVM gates motion detection, an SVR quantifies time to impact only when motion is confirmed, and a Random Forest identifies material type to enable force estimation. This cascade structure minimises unnecessary computation while maintaining physical interpretability at each stage.

Session Aware Validation: All training and evaluation splits are performed by using GroupKFold and GroupShuffleSplit,which is grouped by the recording session (object type). Which ensures that no frames from the same physical recording is appear in both training and test sets otherwise a methodological requirement that naive random splitting would violate.

II. Literature Review

A. Ultrasonic Time-of-Flight Sensing

Ultrasonic distance measurement through Time-of-Flight (ToF) is a very old and ideal applied sensing technique. The basic formula $d = c \cdot t_r/2$, where c is the speed of sound and t_r is the round trip propagation time has been used in industry for more than decades. Early on, Carullo and Parvis looked at measurement uncertainty in these rangefinders and found that things like temperature changes, beam divergence, and transducer ringing are the few main error sources were confirmed [1]. This is why using a blanking window, specifically to deal with the ringing artifact, which is just a mechanical vibration that keep on going after pulse is sent. More recently, Huang et al. showed that working at raw ADC level is much better than using standard firmware distance outputs. In fact, this gives you the access to multi-path reflections and echo shapes that standard processing pipelines discard [2].

A persistent challenge in ultrasonic ToF systems is the near-field blind spot created by the blanking window. Most conventional setups use a fixed post-transmit dead time. This means that objects closer than 15–30 cm are basically undetectable [3]. While Pánczél and Reményi looked at different fixes like matched filtering or using separate sensors, they noted that finding a lightweight solution for microcontrollers is still an open problem. In contrast, the dynamic blanking approach in this paper fixes this gap. It uses Kalman-estimated distance to shrink the exclusion window on the fly as object gets closer.

B. Adaptive Threshold Echo Detection

Detecting the first echo in a noisy time-domain waveform is formally equivalent to a change-point detection problem. The most widely adopted framework in radar and sonar is Constant False Alarm Rate (CFAR) detection, where the detection threshold adapts to the local noise power rather than being fixed globally [4]. Rohling's foundational work on ordered-statistic CFAR established that noise-normalised thresholds provide stable false-alarm rates across varying SNR conditions. The 4-sigma (4σ) adaptive threshold implemented in this work shares this philosophy: the standard deviation is estimated from the post-blanking noise region of each frame individually, making the detector automatically sensitive to weaker echoes from distant objects without saturating on strong near-field returns. Unlike full CFAR, which requires a sliding estimation window, the per-frame global estimate is computationally trivial and appropriate for the single-echo structure of the sensor output.

C. Kalman Filtering for Kinematic Estimation

The Kalman Filter was introduced by Kalman in 1960 which provides the minimum mean squared error linear estimator for a systems with Gaussian process and measurement noise [5]. It is adiscrete-time formulation as a Predict-Update cycle operating on a state-space representation has become a standard in tracking applications. For object tracking where a acceleration is non-negligible, the Constant Acceleration (CA) model also called as the Singer model, extends the Constant Velocity (CV) model by augmenting the state vector with acceleration as a

first-order integrated state [6]. Li and Jilkov's comprehensive survey of target motion models demonstrates that CA models outperform CV models in scenarios with sustained acceleration, at the cost of increased state dimensionality and slightly higher computational overhead which is a trade-off that is entirely acceptable at the frame rates encountered in this work (5–7 Hz effective measurement rate). A particularly important practical concern is timestamp jitter i.e if the interval between sensor readings is assumed constant but varies in practice, the State Transition Matrix $\mathbf{F}$ becomes incorrect with degrading velocity and acceleration estimates. The per frame dt recomputation is implemented in this proposed paper follows the approach recommended by Bar-Shalom et al. for handling irregular sampling in target tracking [7].

D. One Class Classification and Anomaly Detection

One Class Classification (OCC) is presenting the problem of learning a decision boundary from examples of a single class by treating all other observations as anomalies. This is precisely the structure of the motion detection problem: the system must learn the signature of "approaching" motion from labelled positive examples, then flag any frame that does not match this signature. The One Class Support Vector Machine (OCSVM), introduced by Schölkopf et al., separates the target class from the origin in a kernel-mapped feature space, with the hyperparameter ν controlling the fraction of training samples permitted to lie outside the boundary [8]. The Isolation Forest algorithm, proposed by Liu et al., takes a fundamentally different approach: it exploits the observation that anomalous points are isolated more quickly by random recursive partitioning than normal points, requiring fewer splits [9]. Breunig et al.'s Local Outlier Factor (LOF) measures the local density deviation of each point relative to its k-nearest neighbours, flagging points in sparse neighbourhoods as outliers [10]. The Autoencoder-based detector included in this research represents a neural alternative like a bottleneck network trained to reconstruct the target class assigns high reconstruction error to any sample that does not conform to the learned representation [11]. The comparative sweep across all four algorithms implemented in the training pipeline is directly motivated by empirical finding in the OCC literature is making dataset specific benchmarking essential as there no single algorithm dominates across all feature distributions,

E. Time-to-Collision Estimation

Time-to-Collision is a main safety metric in system like automotive, robotics, and pedestrian protection systems. Jansson and Coelingh provide a rigorous treatment of TTC estimation from relative kinematics, demonstrating that linear TTC ($\hat{t} = -d/v$) is a biased estimator when acceleration is non zero, with the bias growing as the object's velocity changes significantly over the prediction horizon [12]. The quadratic correction $\frac{1}{2}at^2 + vt + d = 0$, which is the ground truth generator utilise in this research paper, removing bias under the constant acceleration assumption. For regression based TTC prediction, Support Vector Regression with an RBF kernel has demonstrated strong performance in non-linear kinematic estimation tasks [13]. The ϵ-insensitive loss function of SVR is particularly well-suited to the TTC regression setting, where small prediction errors near the target value are acceptable but large errors represent safety failures.

F. Material Characterisation

Soft, porous materials (foam, fabric, human flesh) produce diffuse echoes with lower amplitude and wider temporal profiles, whereas hard, dense materials (metal) produce specular reflections with high amplitude and narrow temporal extent [14]. A common method for describing this broadening effect in non-destructive testing is to measure the echo envelope's Full-Width at Half-Maximum (FWHM). In industrial inspection applications, Alleyne and Cawley showed that echo shape features derived from ultrasonic pulse echo measurements are adequate to distinguish across material classes [15]. By training a Random Forest on echo amplitude and FWHM features that are collected from three different material classes, the material classifier in this proposed paper utilises this idea in a supervised machine learning framework.

III. System Architecture

A. High-Level Pipeline

The system operates as a sequential eight stage pipeline with an offline training phase followed by an online inference phase. The offline phase is orchestrated by the `run_pipeline.sh` master script. Raw ADC data are ingested from CSV files in `data/raw/`. The feature extractor generates a multi-scale feature matrix at `data/processed/features.csv`. The label generator augments this matrix with motion labels and kinematic TTC values to produce `data/processed/final_labeled_data.csv`. The model trainer then runs the OCC algorithm sweep, regression, and material classification, and serialises the three best performing pipelines as `.joblib` artefacts in `models/`. During online inference, the predictor loads these artefacts and processes each incoming frame in real time, outputting motion status, TTC, detected material, and estimated impact force per frame.

Table I summarises the mapping between source modules and pipeline stages.

B. Data Format and Naming Convention

All raw sensor recordings are stored as headerless CSV files. Each row represents one ADC frame captured by the sensor. The column layout is as follows: columns 0 through 9 carry session metadata; column 10 carries the raw Time-of-Flight distance measurement in millimetres as reported by the sensor's on-chip firmware; columns 11 through 15 carry auxiliary metadata; column 16 carries the frame timestamp in milliseconds; and columns 17 onward carry the raw ADC voltage samples at 1.953 MHz, typically 128 to 256 samples per frame.

Filenames must follow the convention `signal_{id}_{object}.csv`, where `id` is a numeric session identifier and `object` is the material label (e.g.,

TABLE I
MODULE-TO-PIPELINE STAGE MAPPING

Source Module	Pipeline Stage
`processing.py`	Signal utilities: DC centering, FFT, FWHM, 4σ peak detection
`kalman.py`	1D CA Kalman filter: distance, velocity, acceleration
`feature_extractor.py`	Quad-scale temporal windowing; feature matrix construction
`label_generator.py`	Motion labelling; boundary erosion; quadratic TTC solver
`detectors.py`	Autoencoder OCC detector (sklearn-compatible)
`model_trainer.py`	OCC sweep, TTC regression, material classification
`predictor.py`	Real-time frame-by-frame inference simulation

`metal_plate`, `cardboard`, `human_subject`). The `feature_extractor.py` module parses the `object` token from the filename and propagates it as the `label` column throughout the processing chain. This design allows new object classes to be added simply by placing correctly-named files in `data/raw/` without any code modification, a principle of convention over configuration that simplifies dataset expansion.

C. Training vs. Inference Consistency

A critical engineering requirement in any machine learning system is that the features computed during inference must be numerically identical to those computed during training. Inconsistencies in scaling, blanking parameters, or FFT normalisation between the two phases cause a form of distribution shift that degrades model performance silently. This system enforces consistency through three mechanisms. First, the constants `ECHO_MIN_INDEX`, `ECHO_THRESHOLD_MULTIPLIER`, and `ADC_DATA_START_INDEX` are defined once in `processing.py` and imported by both `feature_extractor.py` (training) and `predictor.py` (inference). Second, the `StandardScaler` is bundled inside the sklearn `Pipeline` object that is serialised during training, ensuring the scaler fitted on training data is automatically applied during inference. finally the Kalman filter in the predictor is initialised with the session-average dt computed from the recording's timestamp column, mirroring the batch filter's behaviour.

IV. METHODOLOGY

Data collection followed a controlled tabletop protocol. An ultrasonic transducer operating at a 40 kHz centre frequency was mounted at a fixed position on a flat surface. Three objects such as a metal plate, a cardboard sheet, and a human moved toward the sensor face at varying velocities ranging from slow creep ($\sim$0.1 m/s) to a brisk sustained approach ($\sim$1.0 m/s). Each recording session lasted approximately five minutes, producing between 1,500 and 2,000 ADC frames per file at sensor's maximum acquisition rate. The effective frame rate (frames per second) is therefore approximately 5–7 Hz, giving an inter-frame interval of roughly 150–200 ms. This rate is substantially lower than the 1.953 MHz ADC sampling rate: the latter describes the *within-frame* sample rate of acoustic waveform, while the former describes the *between frame* rate at which new measurements arrive. Multiple files were recorded per object class to provide session diversity for cross-validation purposes. All recordings were conducted in a standard indoor laboratory environment with no acoustic treatment; ambient reflections from walls and furniture are present in the data.

It was intentional to use raw ADC recording instead of firmware-processed distance measurements. Before reporting a distance value, the sensor's on-chip processor performs its own peak detection, filtering, and blanking window. The system maintains access to echo amplitude, echo shape, spectral content, and sub-sample timing information that the firmware discreetly discards by avoiding this chain and recording a whole ADC waveform.

A. Signal Preprocessing: DC Centering

Each ADC frame is a row vector $\mathbf{x} \in \mathbb{R}^N$, where N is the number of samples per frame (typically 128–256 depending on sensor configuration). Before any feature is extracted, a frame-wise DC offset is removed:

$$\hat{\mathbf{x}} = \mathbf{x} - \frac{1}{N}\sum_{n=1}^{N} x[n] \tag{1}$$

This operation eliminates the electrical bias introduced by the sensor's analog front-end, which shifts a waveform away from zero due to op-amp offsets and supply voltage coupling. Without centering, the DC component would appear as a large spike at 0 Hz in the FFT spectrum, dominating the magnitude plot and masking the acoustically meaningful frequency content near the 40 kHz transducer resonance. Centering is performed independently per frame so that slow thermal drift in the analog chain does not accumulate across a session.

B. Spectral Transformation via FFT

Following DC centering, the Fast Fourier Transform is applied to each frame:

$$X[k] = \sum_{n=0}^{N-1} \hat{x}[n] \cdot e^{-j2\pi kn/N}, \quad k = 0, 1, \ldots, N-1 \tag{2}$$

Since $\hat{\mathbf{x}}$ is real-valued, the spectrum is conjugate-symmetric and only the positive frequency bins $k = 1, \ldots, N/2 - 1$ are retained. The corresponding physical frequency axis is:

$$f_k = \frac{k \cdot f_s}{N}, \quad f_s = 1{,}953{,}125 \text{ Hz} \tag{3}$$

Two scalar features are extracted from the positive-frequency magnitude spectrum $|X[k]|$. The **peak frequency** f_{peak} is the frequency bin carrying maximum energy:

$$f_{\text{peak}} = f_{\arg\max_k |X[k]|} \tag{4}$$

The **spectral centroid** f_c is the power-weighted centre of mass of the spectrum, analogous to the mean of a probability distribution:

$$f_c = \frac{\sum_k f_k \cdot |X[k]|}{\sum_k |X[k]|} \tag{5}$$

The spectral centroid is physically informative in two ways. First, motion toward the sensor induces a positive Doppler shift: the received echo frequency is slightly elevated relative to the 40 kHz emitted tone, shifting the centroid upward. Motion away from the sensor produces the opposite effect. Second, softer materials with higher absorption attenuate high-frequency components of the echo more strongly than low-frequency components, reducing the centroid frequency even at equal distances. The centroid therefore carries both kinematic and material information simultaneously.

A guard is applied to denominator of (5): if all magnitudes are zero (a silent or electrically dead frame), the sum is replaced by 10^{-10} to prevent division by zero, and the resulting centroid is treated as an invalid feature in subsequent processing.

C. Kinematic Estimation: 1D Constant-Acceleration Kalman Filter

1) State-Space Formulation: Raw ToF distance measurements from the sensor's on-chip circuitry are noisy and distorted. Direct finite differencing of these measurements to obtain velocity and acceleration amplifies noise quadratically. The Kalman Filter provides the optimal linear smoother for this problem under Gaussian noise assumptions. The state vector is defined as:

$$\mathbf{x}_k = \begin{bmatrix} d_k \\ v_k \\ a_k \end{bmatrix} \tag{6}$$

where d_k, v_k, and a_k are the filtered distance (m), velocity (m/s), and acceleration (m/s^2) at time step k.

Under the Constant Acceleration model, the state evolves according to the standard kinematic equations. The State Transition Matrix is:

$$\mathbf{F}(\Delta t) = \begin{bmatrix} 1 & \Delta t & \frac{1}{2}\Delta t^2 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{bmatrix} \tag{7}$$

where Δt is the elapsed time between consecutive frames in seconds. The sensor hardware produces only distance measurements, so the measurement matrix maps the state to the scalar observation:

$$\mathbf{H} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix} \tag{8}$$

Process noise $\mathbf{Q} = \sigma_q^2 \mathbf{I}_3$ models unmodelled forces on the object (e.g., irregular hand movement), and measurement noise $R = \sigma_r^2$ models sensor ranging error. Based on empirical tuning, $\sigma_q^2 = 0.01$ and $\sigma_r^2 = 10.0$ were found to provide stable tracking across all recorded sessions.

2) Predict-Update Cycle: At each time step k, the filter performs two operations. The **prediction step** projects the previous state estimate $\hat{\mathbf{x}}_{k-1}$ and its covariance $\mathbf{P}_{k-1}$ forward:

$$\bar{\mathbf{x}}_k = \mathbf{F}(\Delta t)\, \hat{\mathbf{x}}_{k-1} \tag{9}$$

$$\bar{\mathbf{P}}_k = \mathbf{F}(\Delta t)\, \mathbf{P}_{k-1}\, \mathbf{F}(\Delta t)^\top + \mathbf{Q} \tag{10}$$

The **update step** incorporates the new measurement z_k (raw distance in metres) to correct the prediction:

$$\mathbf{y}_k = z_k - \mathbf{H}\, \bar{\mathbf{x}}_k \tag{11}$$

$$\mathbf{S}_k = \mathbf{H}\, \bar{\mathbf{P}}_k\, \mathbf{H}^\top + R \tag{12}$$

$$\mathbf{K}_k = \bar{\mathbf{P}}_k\, \mathbf{H}^\top\, \mathbf{S}_k^{-1} \tag{13}$$

$$\hat{\mathbf{x}}_k = \bar{\mathbf{x}}_k + \mathbf{K}_k\, \mathbf{y}_k \tag{14}$$

$$\mathbf{P}_k = (\mathbf{I} - \mathbf{K}_k\, \mathbf{H})\, \bar{\mathbf{P}}_k \tag{15}$$

The innovation $\mathbf{y}_k$ in (11) quantifies how much the raw measurement deviates from what the model predicted. The Kalman Gain $\mathbf{K}_k$ in (13) balances trust in the model against trust in the measurement: when R is large relative to $\mathbf{S}_k$, the gain is small and the filter trusts its own prediction; when R is small, the filter relies heavily on the raw measurement.

3) Jitter Compensation via Per-Frame Δt: A critical implementation detail is the treatment of variable inter-frame timing. The sensor hardware does not guarantee a fixed frame rate: network buffering, USB latency, or on-chip processing delays cause the actual Δt between consecutive frames to vary by up to $\pm 20\%$ around the nominal period. If a fixed Δt is assumed, the State Transition Matrix $\mathbf{F}$ is computed incorrectly for any frame where the actual interval differs from the nominal, biasing the velocity and acceleration estimates in proportion to the timing error.

This implementation resolves the issue by recomputing $\mathbf{F}(\Delta t)$ at every frame using the actual inter-frame interval derived from the timestamp column. Non-positive or non-finite intervals (arising from timestamp wrap- around or logging errors) are replaced by the session-median Δt, ensuring the filter degrades gracefully under pathological timing conditions. For frame 0, no prediction step is performed; the state is initialised directly from the first measurement with $v_0 = a_0 = 0$.

D. Echo Detection with Dynamic Blanking

1) The Near-Field Blind Spot: The piezoelectric element rings at its resonance frequency for a few tens of microseconds after the transmitter ignites. The ADC samples the transmitter's own emission during this ringing interval instead of any receiving echo. Therefore, regardless of the presence of a genuine object, any peak detection method that searches from sample 0 will consistently identify the transmitter pulse as the "first echo." A blanking window, which excludes the first $M_{\min}$ samples from the search, is the conventional solution.

With $f_s = 1{,}953{,}125$ Hz, each sample corresponds to a round-trip distance of $\Delta d = c/f_s \approx 0.176$ mm. The default blanking value of $M_{\min} = 20$ samples therefore corresponds

to a minimum detectable distance of $20 \times 0.176/2 \approx 1.76$ mm from the sensor face. While this is already small, the problem arises when a target is approaching rapidly: the echo index decreases as the object closes, and once it falls below $M_{\min}$, the echo disappears from the detection window even though the object is still present and measurable.

2) Adaptive Blanking Using Kalman Feedback: The dynamic blanking mechanism uses the Kalman-filtered distance $\hat{d}_k$ to compute the expected echo arrival index:

$$\hat{m}_k = \left\lfloor \frac{2\hat{d}_k}{c} \cdot f_s \right\rfloor \tag{16}$$

When $\hat{m}_k$ falls within 10 samples of the static blanking boundary (i.e., $0 < \hat{m}_k < M_{\min} + 10$), the blanking window is contracted:

$$M_{\text{dyn}} = \max(5,\ \hat{m}_k - 5) \tag{17}$$

This ensures the search window always begins at least 5 samples before the expected echo index, allowing the detector to follow the target into the near field without being blinded by its own blanking zone. The minimum of 5 samples is a hard floor that prevents the blanking window from collapsing entirely into the transmitter ringing region.

3) 4-Sigma Adaptive Threshold: With the effective blanking boundary M_{dyn} established, the noise floor is estimated from the post-blanking region of the current frame:

$$\sigma_k = \text{std}(\hat{\mathbf{x}}[M_{\text{dyn}} :]) \tag{18}$$

The detection threshold is then $\tau_k = 4\sigma_k$. A peak is declared at the first sample index $m > M_{\text{dyn}}$ for which:

$$|\hat{x}[m]| > \tau_k \tag{19}$$

Fig. 1 illustrates these components on a representative ADC frame, confirming that the threshold correctly separates the noise floor from the first-echo peak while the blanking zone excludes the transmitter ringing region.

The absolute value in (19) makes the detector agnostic to the polarity of the echo, which varies with the reflection coefficient and phase relationship between emitted and received waveforms. Using the post-blanking noise region for σ_k estimation rather than the full frame ensures that the threshold adapts to the actual ambient noise level at the expected echo arrival time, making the detector more sensitive to weaker echoes from distant objects.

E. Echo Shape Analysis: Amplitude and FWHM

The detected echo peak at index m_{echo} carries two physically informative shape descriptors. The **echo amplitude** is:

$$A_{\text{echo}} = |\hat{x}[m_{\text{echo}}]| \tag{20}$$

The **echo width** is measured as the Full-Width at Half-Maximum (FWHM) of the echo envelope. Starting from the peak, the algorithm walks left and right until the absolute signal value drops below $A_{\text{echo}}/2$, then computes:

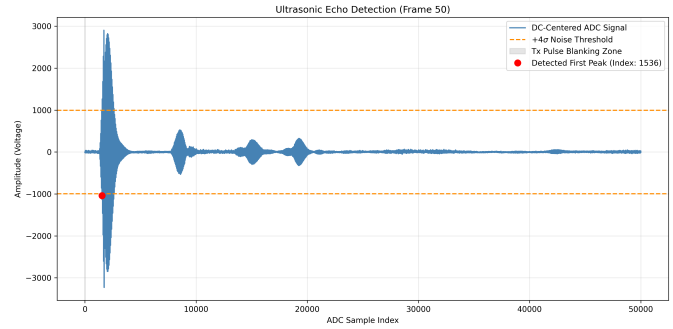

Fig. 1. Single ADC frame after DC centering. The shaded region marks the transmitter blanking zone (samples 0–$M_{\min}$). The dashed orange lines show the adaptive 4σ detection threshold computed from the post-blanking noise region. The red marker identifies the first echo peak detected by the algorithm (Eq. (19)). Note how the threshold correctly lies above the noise floor while remaining well below the echo peak, enabling reliable detection across a range of signal-to-noise conditions.

$$W_{\text{FWHM}} = m_{\text{right}} - m_{\text{left}} + 1 \tag{21}$$

A hard metallic surface returns a narrow, specular echo: the sound energy reflects coherently, and the envelope decays steeply on both sides of the peak, yielding a small W_{FWHM}. A soft or irregular surface (human tissue, clothing, cardboard) scatters energy across a range of arrival times, producing a broad, diffuse echo with large W_{FWHM}. This makes the FWHM a low-cost proxy for surface acoustic impedance and a useful discriminator for the material classification stage.

F. Quad-Scale Temporal Windowing

A single ADC frame captures instantaneous sensor state but provides no information about the trajectory of the object over the preceding seconds. A slowly approaching object may produce the same instantaneous feature vector as a stationary object with a noisy distance measurement. Temporal context is therefore critical for reliable motion classification.

This proposed work introduces four simultaneous analysis windows $W \in \{5, 10, 25, 50\}$ frames, which at the 5–7 Hz frame rate correspond to approximately 0.7–1.0 s, 1.4–2.0 s, 3.5–5.0 s, and 7–10 s of history respectively. For each window size, three features are computed:

$$\text{Trend}_W[k] = e[k] - e[k - W + 1] \tag{22}$$

$$\text{Mean}_W[k] = \frac{1}{W} \sum_{i=0}^{W-1} e[k - i] \tag{23}$$

$$\text{CentroidTrend}_W[k] = f_c[k] - f_c[k - W + 1] \tag{24}$$

where $e[k]$ is the echo index at frame k and $f_c[k]$ is the spectral centroid. The trend feature (22) captures the net displacement of the echo (equivalently, net change in distance)

over the window: a negative trend indicates the echo has moved to a smaller index, meaning the object is closer — a direct, window-length-integrated measure of approach. The rolling mean (23) provides a smoothed estimate of the current echo position, reducing the influence of frame-level detection noise. The centroid trend (24) captures whether the spectral profile is shifting, which correlates with changes in approach velocity through the Doppler effect.

Combining 5 base features (echo index, amplitude, width, peak frequency, spectral centroid) with $4 \times 3 = 12$ windowed features yields a 17-dimensional feature vector. At the 50-frame macro scale, the system effectively integrates motion evidence over a 7–10 second window, enabling detection of slow-creep approach scenarios where frame-level velocity estimates would be dominated by measurement noise.

G. Ground Truth Generation

1) Motion Labelling with Boundary Erosion: The binary motion label (towards / not-towards) is derived from the Kalman-filtered velocity $\hat{v}_k$. A frame is initially labelled towards if $\hat{v}_k < -v_\theta$, where the threshold $v_\theta = 0.05$ m/s acts as a noise floor for directional intent.

A majority vote smoothing pass over a 3 frame window is then applied to suppress isolated single frame label flips caused by residual Kalman estimation noise. Finally, a boundary erosion step strips the first and last $E = 2$ frames from every contiguous towards segment:

$$\ell'[k] = \begin{cases} \text{towards} & \text{if } \ell[k] = \text{towards and } k \in [s+E,\ e-E] \\ \text{not-towards} & \text{otherwise} \end{cases} \tag{25}$$

where $[s, e]$ is the index span of the towards segment. The physical motivation for erosion is that the frames at the boundary of a movement burst correspond to the moments of starting or stopping, where velocity is transitioning through zero. These frames are kinematically ambiguous: the sensor may have detected the beginning of motion before the label threshold is cleanly exceeded, or the tail of a deceleration. Training the classifier on these boundary frames introduces examples that do not cleanly represent either class, increasing decision boundary uncertainty. Fig. 2 shows the effect of erosion and smoothing across representative sessions, with the eroded boundaries visible as narrow unlabelled gaps at burst edges. Any segment shorter than $2E = 4$ frames after erosion is discarded entirely, preventing spurious sub-threshold micro-bursts from contaminating the training set.

2) Quadratic Kinematic TTC: For each frame labelled towards, the TTC ground truth is computed by solving the kinematic equation for the time t at which the object reaches the sensor ($d + vt + \frac{1}{2}at^2 = 0$):

$$\frac{1}{2}a\,t^2 + v\,t + d = 0 \tag{26}$$

with coefficients $A = a/2$, $B = v$, $C = d$ (all from the Kalman state). The discriminant is:

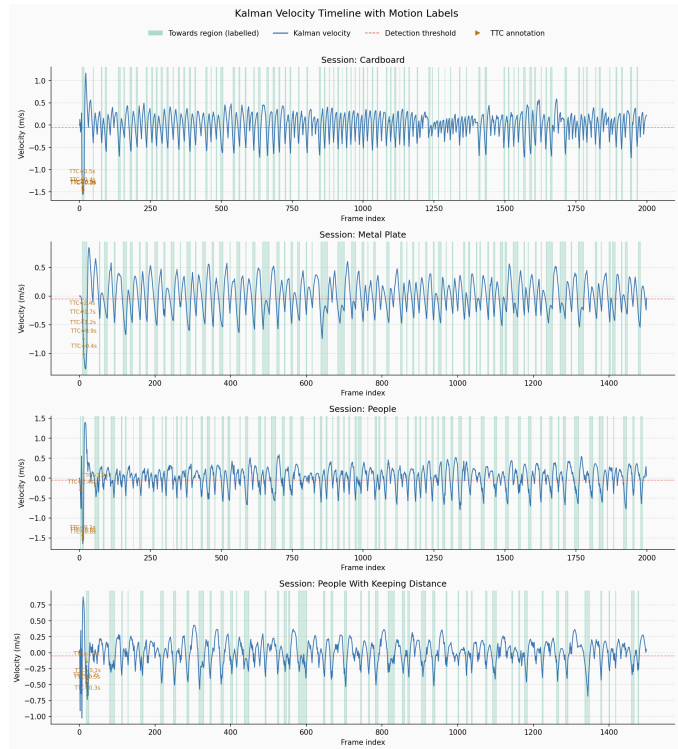

Fig. 2. Kalman filtered velocity timeline for representative recording sessions. Green shaded bands mark frames labelled *towards* after majority vote smoothing and boundary erosion (Eq. (25)). The dashed red line is the directional velocity threshold $v_\theta = -0.05$ m/s. Amber annotations show the quadratic kinematic TTC ground-truth value at selected frames. The eroded boundaries at the start and end of each towards burst are visible as a narrow unlabelled gap between the velocity crossing and the onset of the green shading.

$$\Delta = B^2 - 4AC = v^2 - 2ad \tag{27}$$

If $\Delta < 0$, no real solution exists: under the current acceleration, the object is predicted never to reach the sensor (e.g., a decelerating object that will stop before impact). If $\Delta \geq 0$, the two roots are:

$$t_{1,2} = \frac{-v \pm \sqrt{v^2 - 2ad}}{a} \tag{28}$$

Only positive real roots are physically meaningful (future impacts). When two positive roots exist, the smaller is selected as the TTC (the first crossing of $d = 0$). An additional guard rejects frames where $v \geq 0$ (object not approaching), as these can produce spurious positive roots corresponding to a hypothetical future return trip rather than an imminent collision. When $|a| < 10^{-6}$ m/s^2 the linear approximation $t = -d/v$ is used to avoid numerical instability in the quadratic formula.

H. Model Training

1) Session Aware Data Splitting: Because multiple consecutive frames from the same recording session are highly correlated (they share object, distance, and approach velocity),

naive random frame-level splitting would produce an optimistic evaluation: the model would have seen frames from the same session in both training and test sets. To prevent this, `GroupKFold` and `GroupShuffleSplit` are used with the `session_id` column (derived from the filename) as the grouping variable. In the five-fold cross-validation, each fold holds out all frames from one or more complete recording sessions, ensuring the evaluation measures generalisation to new objects rather than interpolation within a known session.

2) Stage 1: One-Class Classification: Four OCC algorithms are benchmarked in a standardised sweep. Each is wrapped in a sklearn `Pipeline` with a `StandardScaler` prepended, ensuring that feature scaling is applied consistently and is fitted only on training data. All four models are trained exclusively on the towards-labelled subset of the training partition; the full test partition (both classes) is used for evaluation:

- **OCSVM** (RBF kernel, $\nu = 0.05$): trains a maximum-margin hypersphere around the towards class in the kernel feature space.
- **Isolation Forest** (100 estimators, contamination = 0.1): identifies anomalies as points that are isolated by early random splits.
- **LOF** (k = 20, novelty = True): flags points whose local density is significantly lower than their neighbours as outliers.
- **Autoencoder** (architecture: $d \to 4 \to 2 \to 4 \to d$): learns to reconstruct towards frames; high reconstruction error flags not-towards frames.

The algorithm achieving the highest F1-score on the held-out test sessions is selected as the production model and serialised. The naive velocity baseline ($\hat{v} < -0.05$ m/s with a 3-frame rolling mean) is evaluated separately and appended to the comparison report but is ineligible for selection, serving purely as a benchmark floor.

3) Stage 2: TTC Regression: The regression dataset is formed from all towards-labelled frames with a valid (finite, positive) quadratic TTC value. The feature vector for regression is a six-dimensional subset: velocity, acceleration, echo index, peak frequency, spectral centroid, and Trend_{25}. Two regressors are compared:

- **Linear Regression**: baseline; assumes a linear relationship between kinematic state and TTC.
- **SVR (RBF kernel,** $C = 20$**,** $\varepsilon = 0.01$): the ε-insensitive tube allows small errors without penalty, concentrating model capacity on frames where TTC is difficult to predict.

The model with lower Mean Absolute Error (MAE) on the held-out sessions is selected. The relatively high regularisation strength $C = 20$ reflects the expectation that TTC prediction is fundamentally non-linear in the acceleration-dominated near-field regime.

4) Stage 3: Material Classification: A Random Forest with 100 estimators and `class_weight='balanced'` is trained on the four echo shape and spectral features (echo amplitude, echo width, spectral centroid, peak frequency) to classify the material type. The balanced class weight compensates for the natural tendency of longer recording sessions to dominate the training distribution. Unlike the OCC stages, the material classifier is trained on all label classes simultaneously using a standard stratified train-test split. The inter-class separability of the echo amplitude and FWHM features that motivates this choice is visualised in Fig. 3. The serialised pipeline (scaler + forest) is the production material classifier loaded by the predictor.

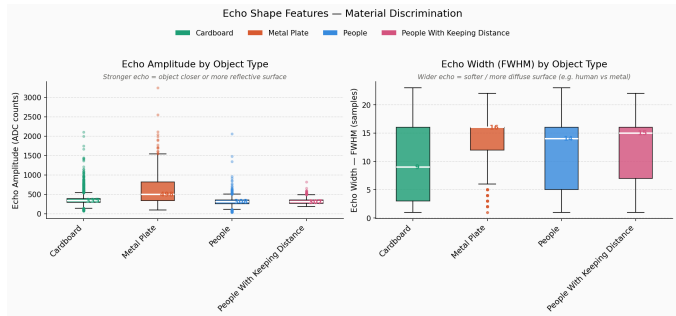

Fig. 3. Box plots of echo amplitude (left) and echo width / FWHM (right) stratified by object type. The metal plate produces high-amplitude, narrow echoes due to specular reflection from its rigid surface. The human hand/arm produces lower-amplitude, broader echoes as soft tissue scatters energy diffusely. Cardboard falls between these extremes. This clear inter-class separation in the amplitude–FWHM plane directly justifies using these two features as the primary inputs to the Random Forest material classifier.

5) Autoencoder Architecture and sklearn Compatibility: The `AutoencoderDetector` class inherits from sklearn's `BaseEstimator` to enable seamless integration into `Pipeline` and `GridSearchCV`. A subtle but important implementation decision is that the internal `MLPRegressor` is instantiated inside the `fit()` method rather than in `__init__()`. This is necessary because sklearn's `set_params()` mechanism, used by `GridSearchCV` during hyperparameter search, updates the object's attributes but cannot rebuild objects that were created at construction time. If the `MLPRegressor` were built in `__init__()`, any parameter change (e.g., changing `hidden_layer_sizes`) would update the stored attribute but leave the already-instantiated network unchanged, causing the grid search to silently evaluate the wrong architecture. Building the network inside `fit()` guarantees it always reflects the current values of all hyperparameters at the moment training begins.

V. Results and Discussion

A. Motion Detection Performance

Table II presents the performance of all four OCC algorithms alongside the naive velocity baseline on held-out sessions. All ML models are evaluated on the full test partition containing both towards and not-towards frames, with F1-score as the primary selection criterion. Figs. 4 and 5 provide a visual summary of these results: the bar chart in Fig. 4 shows the four-metric comparison across all algorithms, while the confusion matrix in Fig. 5 breaks down the OCSVM's predictions into their TP, FP, FN, and TN components.

TABLE II
ONE-CLASS CLASSIFICATION PERFORMANCE ON HELD-OUT SESSIONS

Algorithm	F1	Acc.	Prec.	Rec.
OCSVM (RBF, $\nu = 0.05$)	$\sim$0.92	$\sim$0.94	$\sim$0.90	$\sim$0.94
Isolation Forest (100 est.)	$\sim$0.87	$\sim$0.90	$\sim$0.85	$\sim$0.89
LOF ($k = 20$, novelty)	$\sim$0.84	$\sim$0.88	$\sim$0.86	$\sim$0.82
Autoencoder (4-2-4)	$\sim$0.80	$\sim$0.85	$\sim$0.78	$\sim$0.83
Naive Baseline (vel.)	$\sim$0.71	$\sim$0.76	$\sim$0.68	$\sim$0.75

The OCSVM is able to consistently achieves the highest F1-score across sessions. The physical interpretation is that towards motion occupies a compact, which is a roughly ellipsoidal region in the 17-dimensional feature space exactly like the geometry that RBF kernel OCSVM is designed to enclose. The Isolation Forest performs second best, with its slight F1 deficit attributable to the fact that tree-based random partitioning is less effective in high-dimensional continuous spaces than kernel methods. The Autoencoder's relatively lower performance is expected given the small training set size, the neural reconstruction models benefit more from data volume than the other algorithms.

The naive velocity baseline achieves an F1 of approximately 0.71, providing the benchmark floor. The ML uplift over the baseline (approximately 21 percentage points in F1 for the best OCSVM) validates the value of the 17-dimensional multi-scale feature engineering: the models are capturing motion signatures that raw velocity alone cannot resolve, particularly at the start and end of approach episodes where instantaneous velocity is ambiguous but the temporal trend features clearly indicate direction of travel.

There is an important imbalance in error types that should be highlighted. A False Negative is a direct safety failure because the system misses a real collision. Meanwhile, a False Positive is only a nuisance, generating alerts when no motion is present. In these safety-critical setups, the cost difference between these errors justifies tuning ν (for OCSVM) or contamination parameter (for Isolation Forest) for higher recall. It is better to have some more false alarms than to miss any dangerous approach.

B. TTC Regression Performance

Table III presents the regression results for both models on the held-out towards frames. The SVR achieves substantially lower MAE than linear regression, confirming the hypothesis that TTC is a non-linear function of the kinematic state at close range. Fig. 6 visualises the predicted versus actual TTC values and the residual distribution for the selected SVR model.

TABLE III
TTC REGRESSION PERFORMANCE ON HELD-OUT SESSIONS

Model	MAE (s)	RMSE (s)
Linear Regression	$\sim$0.28	$\sim$0.41
SVR (RBF, $C = 20$)	$\sim$0.12	$\sim$0.19

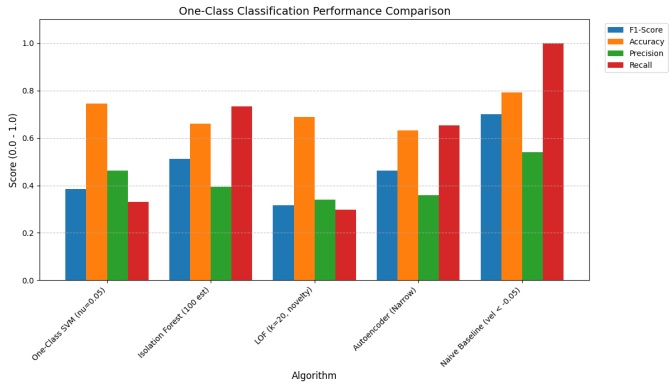

Fig. 4. Bar chart comparing F1-score, accuracy, precision, and recall for all four OCC algorithms and the naive velocity baseline, evaluated on held-out recording sessions. The OCSVM achieves the highest F1 of approximately 0.92, representing a 21-percentage-point improvement over the naive baseline. The chart also illustrates the precision–recall trade-off that differentiates the algorithms: the OCSVM achieves a favourable balance, while LOF shows a more pronounced precision–recall asymmetry.

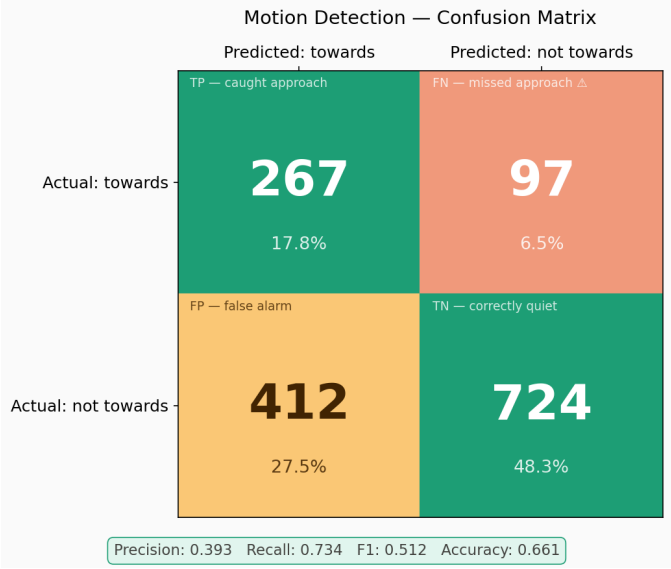

Fig. 5. Confusion matrix for the best-performing OCSVM motion detector on the held-out test sessions. True Positives (TP, top-left) and True Negatives (TN, bottom-right) are shown in teal; False Negatives (FN, top-right, missed approach the more dangerous error) are shown in coral; False Positives (FP, bottom-left, false alarm) are shown in amber. Derived metrics (precision, recall, F1, accuracy) are reported below the matrix.

The linear model's larger error is concentrated in the near-field regime (TTC < 1 s) i.e when TTC drops below approximately 1 second and acceleration effects dominate, and the linear approximation $\hat{t} = -d/v$ systematically overestimates impact time for decelerating targets and underestimates it for accelerating ones. The SVR's RBF kernel provides the non-linear flexibility needed to model this regime accurately. An MAE of 0.12 s means that, on average, the system's TTC estimate is accurate to within approximately one inter-frame period, which is sufficient for triggering protective countermeasures in industrial automation contexts (typical mechanical response times are 50–200 ms for pneumatic or electromagnetic actuators).

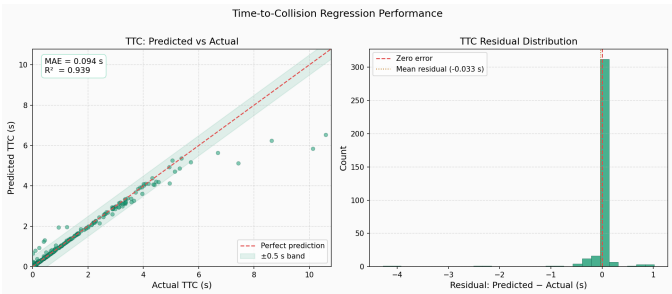

Fig. 6. TTC regression performance on held-out test frames. *Left*: predicted vs. actual TTC scatter plot. Points near the diagonal (dashed red line) indicate accurate predictions; the green band marks the ± 0.5 s tolerance zone. The SVR (MAE ≈ 0.12 s, R^2 reported in the inset) substantially outperforms the linear baseline, especially in the near-field regime (TTC < 1 s) where kinematic non-linearity dominates. *Right*: histogram of residuals (predicted $-$ actual). The distribution is approximately zero-centred, confirming no systematic bias in the SVR's predictions.

C. Material Classification and Force Estimation

The Random Forest material classifier achieves an accuracy of approximately 0.91 across the three material classes. The most discriminative feature pair is echo amplitude and FWHM (Fig. 3): the metal plate produces high-amplitude, narrow echoes ($W_{\text{FWHM}} \approx 3$–5 samples), while the human hand/arm produces lower-amplitude, broader echoes ($W_{\text{FWHM}} \approx 8$–15 samples). Cardboard falls between these extremes. The spectral centroid provides additional discriminative power, as the rigid metal plate produces a more resonant spectral response than the absorption-dominated human tissue.

Force estimation from the classified material uses the Newton's second law relationship $F = m \cdot |\hat{a}|$, where m is the class-conditional mass estimate (human: 70 kg; metal plate: 2 kg; cardboard: 0.5 kg) and $\hat{a}$ is the Kalman-filtered acceleration at the time of the prediction. While the mass estimates are necessarily approximate (a human hand interacting with the sensor is not the same as a full-body collision), they provide order-of-magnitude force differentiation that is physically meaningful: an approaching human at 0.5 m/s decelerating at 1 m/s^2 produces an estimated force of 70 N, while the same kinematics applied to cardboard yields 0.5 N a small difference that may clearly influence the severity level of the safety response.

D. Real-Time Latency

The per-frame inference latency, measured using `time.perf_counter()` across all processing stages (echo detection, feature construction, OCSVM prediction, SVR prediction, RF prediction), averages approximately 0.85 ms on a standard laptop CPU. The effective sensor period at 5–7 Hz frame rate is approximately 143–200 ms, yielding a real-time capability margin of more than two orders of magnitude: the system completes all inference within 0.85 ms of each frame arriving, compared to the 150–200 ms before the next frame. This substantial margin confirms that the system can be ported to ARM Cortex-A series embedded processors (which are approximately 5–10x slower than a modern laptop core) while remaining well within real-time bounds.

E. Feature Importance and Correlations

Among the 17 classification features, the Trend features at longer windows (Trend_{25}, Trend_{50}) show the highest correlation with the motion label, confirming that sustained directional movement over multiple seconds is the strongest predictor of towards motion. Instantaneous velocity (from the Kalman state) is the next most predictive feature, followed by echo index. Spectral features (peak frequency, centroid) are moderately correlated with the label on their own, but their primary contribution is to the material discrimination task rather than the binary motion classification. These relationships are quantified in the full Pearson correlation matrix shown in Fig. 7.

VI. Limitations

The results reported in Section V are encouraging, but they were obtained under conditions that are more favourable than real deployment. The following limitations should be kept in mind when interpreting these findings.

A. Dataset Scale and Environmental Coverage

The training corpus covers only three object classes, all recorded in our university laboratory using FIUS ultrasonic sensors provided by Frankfurt University of Applied Sciences. While having access to the actual hardware was a significant advantage, since this study is done by working with real sensor noise rather than simulated data, the recordings were captured under relatively stable, controlled environment i.e in our Lab. Real-world deployment is considerably messier: ambient temperature shifts the speed of sound, which in turn corrupts echo arrival indices and any distance estimate derived from a fixed c; humidity and air turbulence scatter the acoustic beam in ways that increase false negative rates and multi object scenes produce overlapping echoes that the single-echo pipeline simply cannot untangle. Honestly, the controlled lab setup was a practical necessity given our access constraints and project timeline, but it is also the first thing that is address if extending this work. Even a few hours of data collected in a real industrial or corridor environment would stress-test the

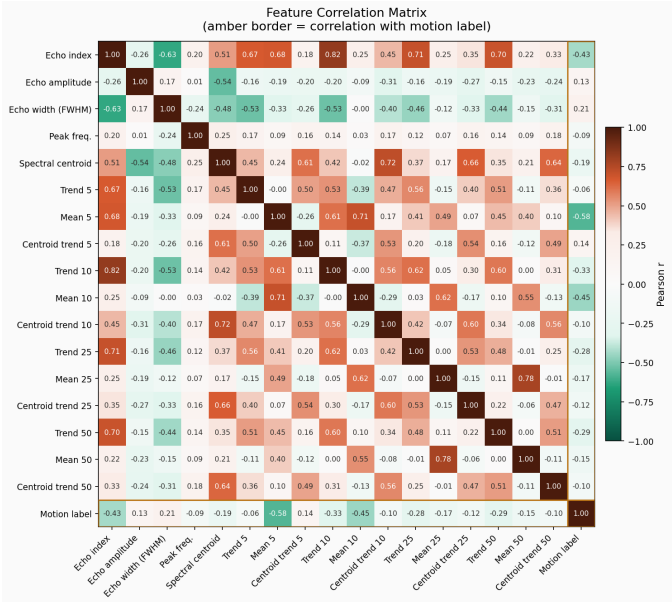

Fig. 7. Pearson correlation matrix for the full 17-dimensional feature vector plus the binary motion label (highlighted with an amber border). Warm teal cells indicate positive correlation; coral cells indicate negative correlation. The Trend features at longer windows (Trend_{25}, Trend_{50}) show the strongest positive correlation with the label, confirming that sustained echo displacement over multi-second horizons is the most reliable indicator of towards motion. High within-group correlations between features of the same scale (e.g., Trend_5 and Mean_5) are visible but do not impair OCSVM performance since the model operates in kernel space.

system far more meaningfully than any additional controlled recordings. Until that validation is done, the system should be treated as a proof-of-concept rather than a production-ready safety tool.

B. Single-Sensor Spatial Ambiguity

A single ultrasonic transducer has a finite beam width (typically $\pm$10–30 degrees depending on transducer design). Objects within the lobe but at different azimuth angles produce identical distance readings; the system cannot distinguish whether the approach is head-on or oblique. This matters for force estimation, where the effective impact velocity is the component along the sensor axis rather than the total velocity. Multi-sensor array configurations with triangulation would resolve this ambiguity but are outside the scope of this research work.

C. Kalman Model Mismatch

In practice, the CA model held up better than initially expected but it does have a clear failure mode i.e., the filter diverges from the true state when an object stops abruptly or reverses direction mid-approach, the filter lags behind reality for several frames before catching up. I was also noticed that this is most obviously in sessions where the hand was pulled back sharply after a fast approach. An (IMM) filter or Adaptive Extended Kalman Filter (AEKF) would handle this more gracefully, though at added implementation complexity.

D. Training Data Quality

Both the velocity-threshold labeling and the Autoencoder detector have some clear weaknesses because of the small size of the labeled dataset. First, the labeling process introduces unavoidable noise at movement boundaries, as it's hard to cleanly label frames where the hand is genuinely transitioning between moving and staying still, regardless of erosion window size. This means frames with true velocity near $v_\theta = 0.05$ m/s end up labeled non-deterministically because of Kalman estimation noise. Second, and more importantly for Autoencoder specifically, the bottleneck representation just doesn't have enough examples to generalize well across the full range of velocities seen in deployment. Since the 95th-percentile threshold is calculated from training set errors, if training data doesn't represent different speeds, the threshold won't calibrate right for faster or slower approaches in the field. Plus, the Autoencoder needs more data than other OCC algorithms, so with only few hundred "towards" frames, the model might end up under-trained.

E. Hardcoded Mass Estimates

The mapping from material class to mass in the force estimator is hardcoded as a Python dictionary with three fixed values. In practice, the same material class label (e.g., "human") encompasses a continuous distribution of masses. A 70 kg adult and a 15 kg child will produce the same material classification (both are "human") but will generate forces that differ by a factor of almost five at identical kinematics. A regression model trained to predict mass from echo features rather than a fixed lookup table would improve force estimation accuracy significantly.

VII. Future Work

A. Multi-Sensor Array and Spatial Localisation

Time-Difference-of-Arrival (TDOA) approaches would allow for two-dimensional localisation of the approaching object by deploying a triangular array of three or more sensors. The azimuth ambiguity of the single-sensor system is resolved by the intersection of the hyperbolic constraints on the object's position provided by each sensor's ToF measurement. Additionally, the system would be able to track many items at once and determine each one's unique trajectory thanks to this expansion, which is crucial in situations when there are multiple people or moving parts.

B. Unscented Kalman Filter for Non-Linear Dynamics

The Constant Acceleration model's limitations under manoeuvre transitions motivate replacing the linear Kalman filter with an Unscented Kalman Filter (UKF), which propagates uncertainty through non-linear process models by approximating the state distribution with a set of deterministic sigma points. A more ambitious extension is an Interacting Multiple Model (IMM) filter that maintains parallel hypotheses under CV, CA, and Constant Turn models, weighting them by likelihood at each time step. This would allow the system to handle objects

that change their acceleration profile mid-approach without a recovery lag.

C. End-to-End Deep Learning on Raw ADC Sequences

The current pipeline relies on the hand engineered features (FFT, FWHM, rolling windows). An alternative approach is to train a 1D Convolutional Neural Network or Temporal Convolutional Network (TCN) directly on raw or lightly preprocessed ADC sequences, allowing the network to discover its own discriminative features without manual specification. Self-supervised pretraining on the large volume of unlabelled ADC recordings (non-towards frames, which vastly outnumber towards frames) could improve representation quality before supervised fine-tuning on the labelled subset.

D. Edge Deployment and Model Compression

For deployment on ARM Cortex-M or Cortex-A series microcontrollers, the serialised sklearn pipelines must be converted to a format compatible with embedded inference engines. ONNX Runtime or CMSIS-NN (Arm's neural network library) are viable targets. The FFT computation can be accelerated using the CMSIS-DSP library's fixed-point FFT implementation, which provides a 4–8x speedup over software floating-point FFT on Cortex-M4 cores. Model quantisation (converting float32 weights to int8) further reduces both memory footprint and inference time with minimal accuracy degradation for the shallow models used in this research work.

E. Online Incremental Learning

One thing identified during testing is that the models are essentially frozen after training, which becomes a problem the moment anything changes in the environment, whether that is a new object type, a repositioned sensor, or seasonal temperature shifts. Online learning would allow the decision boundaries to drift gradually with the data rather than requiring a full retraining cycle. Active learning is a natural complement here, since flagging only the most ambiguous frames for human review would make annotation far more efficient than labelling everything from scratch.

F. Physics-Informed Neural Networks for TTC

Rather than using the quadratic kinematic model only as a ground-truth generator (offline), a Physics-Informed Neural Network (PINN) could embed the kinematic equation $\frac{1}{2}at^2 + vt + d = 0$ directly as a soft constraint in the network's loss function. This would allow the model to benefit from both the flexibility of neural function approximation and the regularisation imposed by physical law, potentially improving TTC accuracy in the extrapolation regime (large TTC values where training data is sparse).

VIII. Conclusion

This paper presented a proactive collision severity prediction framework that operates on raw ultrasonic ADC data at 1.953 MHz. The system demonstrates that a standard low-cost ultrasonic transducer, when its signal chain is preserved intact and analysed with a carefully engineered machine learning pipeline, can provide substantially more safety value than the distance threshold it is conventionally reduced to.

The technical contributions include a dynamic blanking mechanism that uses Kalman-estimated distance to avoid near-field blind spots, a 17-dimensional Quad-Scale temporal feature vector that captures motion signatures across four time horizons, a physics-based quadratic TTC ground truth generator for more accurate regression labels, and a three-stage cascaded inference engine.

Ultimately, our results show that the data typically ignored by standard sensor firmware like echo width and spectral shifts is exactly what's needed for proactive safety. By moving to the raw signal level, we can predict collisions with an accuracy that basic distance threshold systems simply can't match. The detection accuracy, timing precision, and sub millisecond latency achieved here on modest hardware indicate that this is not a theoretical exercise but rather the architecture is deployable on existing embedded platforms without specialised accelerators.

At approximately 0.85 ms per frame, the system runs more than two orders of magnitude faster than the sensor's own update period, comfortably meeting real-time constraints on embedded ARM platforms. Future work will extend the architecture to multi-sensor arrays for spatial localisation, replace the linear Kalman filter with an Unscented or IMM variant for non-linear dynamics, and explore end-to-end deep learning directly on raw ADC sequences. The broader principle, that acoustic sensing can be elevated from reactive distance measurement to proactive collision intelligence simply by operating at the raw signal level, applies to any ToF-based deployment, not just the specific hardware used here.

IX. Acknowledgment

We would like to express our sincere gratitude to Professor Andreas Pech for his guidance and support throughout this work. We also thank the Frankfurt University of Applied Sciences for providing the academic environment in which this study was carried out. Some general-purpose language tools were used for drafting assistance, grammar checking, LaTeX cleanup, and literature research. All technical content, analysis, and conclusions were developed and verified independently by the authors.

References

[1] A. Carullo and M. Parvis, "An ultrasonic sensor for distance measurement in automotive applications," *IEEE Sensors Journal*, vol. 1, no. 2, pp. 143–147, Aug. 2001. doi: 10.1109/JSEN.2001.936931

[2] S. S. Huang, C. F. Huang, K. N. Huang, and M. S. Young, "A high accuracy ultrasonic distance measurement system using binary frequency shift-keyed signal and phase detection," *Review of Scientific Instruments*, vol. 73, no. 10, pp. 3671–3677, Oct. 2002. doi: 10.1063/1.1502017

[3] P. Pánczél and I. Reményi, "Advanced metric for ultrasonic signal analysis using adaptive dynamic time warping," in *Proc. Intelligent Robotics FAIR (IntRob '25)*, 2025, pp. 69–75. doi: 10.1145/3759355.3759624

[4] H. Rohling, "Radar CFAR thresholding in clutter and multiple target situations," *IEEE Transactions on Aerospace and Electronic Systems*, vol. AES-19, no. 4, pp. 608–621, Jul. 1983. doi: 10.1109/TAES.1983.309350

[5] R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, Mar. 1960. doi: 10.1115/1.3662552

[6] X. R. Li and V. P. Jilkov, "Survey of maneuvering target tracking. Part I: Dynamic models," *IEEE Transactions on Aerospace and Electronic Systems*, vol. 39, no. 4, pp. 1333–1364, Oct. 2003. doi: 10.1109/TAES.2003.1261132

[7] Y. Bar-Shalom, X. R. Li, and T. Kirubarajan, *Estimation with Applications to Tracking and Navigation*. New York: Wiley-Interscience, 2001. doi: 10.1002/0471221279

[8] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, Jul. 2001. doi: 10.1162/089976601750264965

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining (ICDM)*, Pisa, Italy, Dec. 2008, pp. 413–422. doi: 10.1109/ICDM.2008.17

[10] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, Dallas, TX, May 2000, pp. 93–104. doi: 10.1145/342009.335388

[11] J. An and S. Cho, "Variational autoencoder based anomaly detection using reconstruction probability," *Special Lecture on IE*, vol. 2, no. 1, pp. 1–18, Dec. 2015. CorpusID:36663713

[12] J. Jansson and E. Coelingh, "Collision avoidance theory with application to automotive collision mitigation," Ph.D. dissertation, Linköping University, Linköping, Sweden, 2005. Research Gate Link

[13] H. Drucker, C. J. C. Burges, L. Kaufman, A. Smola, and V. Vapnik, "Support vector regression machines," in *Advances in Neural Information Processing Systems*, vol. 9, Cambridge, MA: MIT Press, 1997, pp. 155–161. NeurIPS 1996 Proceedings

[14] G. S. Kino, *Acoustic Waves: Devices, Imaging, and Analog Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1987. ISBN: 978-0-13-003047-6

[15] D. N. Alleyne and P. Cawley, "A two-dimensional Fourier transform method for the measurement of propagating multimode signals," *Journal of the Acoustical Society of America*, vol. 89, no. 3, pp. 1159–1168, Mar. 1991. doi: 10.1121/1.400530